

ISO/IEC 27001:2022

Controles 8.25, 8.28 y 8.29

Equipo 5 — IS-722 Calidad de Software

Universidad Nacional de Moquegua — 2026-I

La norma **ISO/IEC 27001:2022** establece los requisitos para establecer, implementar y mantener un **Sistema de Gestión de Seguridad de la Información (SGSI)**. Es la norma internacional de referencia para proteger la confidencialidad, integridad y disponibilidad de la información en cualquier organización.

Anexo A — Controles tecnológicos:

- La norma tiene 93 controles organizados en 4 temas: organizacionales, personas, físicos y tecnológicos.
- El tema **A.8 (Controles tecnológicos)** tiene 34 controles, de los cuales los más relevantes para desarrollo de software son el **8.25, 8.28 y 8.29**.
- Estos tres controles forman un cluster que cubre todo el ciclo de vida del desarrollo seguro.

SGSI

Sistema de Gestión de Seguridad de la
Información

El control **8.25** actúa como paraguas que establece las reglas generales. El **8.28** se enfoca en cómo se escribe el código. El **8.29** verifica que todo funcione antes de producción. Juntos cubren desde la planificación hasta el mantenimiento.

8.25

Secure Development Life Cycle

Establece reglas para el desarrollo seguro de software y sistemas. Aplica durante todo el ciclo de vida: requisitos, diseño, desarrollo, pruebas, despliegue y mantenimiento. Incluye separación de entornos, control de versiones y competencia del equipo.

8.28

Secure Coding

Define principios de codificación segura para reducir vulnerabilidades. Cubre validación de entradas, manejo de secretos, mínimo privilegio, dependencias seguras y manejo de errores. Aplica a código interno y externo.

8.29

Security Testing

Requiere pruebas de seguridad durante el desarrollo y antes de producción. Incluye SAST, DAST, revisiones de código, escaneo de vulnerabilidades y acceptance testing con criterios claros de aceptación.

Un **Secure SDLC** es un ciclo de vida de desarrollo de software con seguridad integrada en cada fase. El concepto clave es **Shift Left**: mover las actividades de seguridad hacia las fases iniciales del desarrollo, donde es más barato corregir problemas.

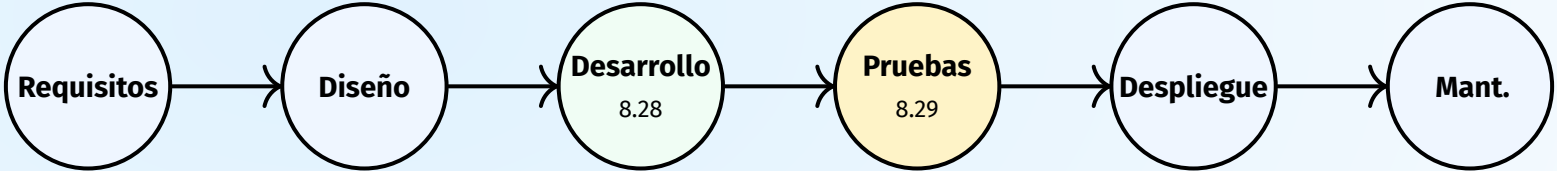
Modelo tradicional

En el enfoque tradicional, la seguridad se revisaba al final del proyecto, después de que el código ya estaba construido. Esto provocaba que las vulnerabilidades se detectaran tarde, con un costo de corrección mucho mayor y más componentes afectados.

SSDLC – Shift Left

Con el SSDLC, la seguridad está presente desde la fase de **Requisitos**. Cada fase del ciclo de vida incluye actividades de seguridad: threat modeling en diseño, codificación segura en desarrollo, pruebas automáticas en CI/CD y hardening en despliegue.

El siguiente diagrama muestra las seis fases del SSDLC y cómo el control 8.25 las cubre. Las fases 3 y 4 están directamente vinculadas a los controles 8.28 y 8.29 respectivamente.



Todos bajo el control **8.25**

Reglas generales del ciclo de vida seguro.

Fase 1 — Requisitos de Seguridad: Se identifican los activos de información que requieren protección (bases de datos, credenciales, código fuente, registros de auditoría). Se clasifican según su sensibilidad (Pública, Interna, Confidencial, Restringida). Se definen los requisitos CIA (Confidencialidad, Integridad, Disponibilidad) más Trazabilidad.

Fase 2 — Diseño Seguro: Se realiza threat modeling usando la metodología **STRIDE** (Spoofing, Tampering, Repudiation, Information Disclosure, DoS, Escalada de privilegios). Se elaboran diagramas de flujo de datos (DFD) para identificar procesos, flujos, almacenes y entidades externas. Se definen los límites de confianza entre capas: entorno externo → aplicación → servicios internos → almacenamiento.

Fase 3 — Desarrollo (8.28): Se aplica codificación siguiendo los 7 principios de seguridad. Se realizan revisiones de código por pares. Se usan herramientas de análisis estático (SAST) integradas en el IDE.

Fase 4 — Pruebas (8.29): Se ejecutan pruebas SAST y DAST. Se realizan penetration tests. Se definen criterios de aceptación: 0 vulnerabilidades críticas, 0 altas, cobertura SAST $\geq 80\%$, dependencias sin CVEs críticos. Si no se cumplen, no se despliega.

Fase 5 — Despliegue: Se aplica hardening (imágenes mínimas, usuarios no root, puertos restringidos). Se gestiona secretos con variables de entorno y gestores centralizados. Se asegura la canalización CI/CD con Security Gate.

La metodología **STRIDE** permite identificar sistemáticamente las amenazas en cada fase de diseño. Para cada componente del sistema se preguntan las 6 categorías de amenazas:

Amenaza	Significado	Pregunta clave
Spoofing	Suplantación de identidad	¿Quién es realmente el usuario?
Tampering	Manipulación de datos	¿Pueden alterarse los datos en tránsito?
Repudiation	Negación de acciones	¿Se puede demostrar quién hizo qué?
Info Disclosure	Exposición de información	¿Se puede leer información confidencial?
DoS	Denegación de servicio	¿Se puede interrumpir el servicio?
EoP	Escalada de privilegios	¿Se pueden obtener permisos extra?

El control 8.28 establece que la organización debe definir principios de codificación segura aprobados. Estos principios deben aplicarse a todo el código: desarrollo interno, externo, bibliotecas y dependencias. El objetivo es reducir la superficie de ataque desde la fase de implementación.

1. Nunca confiar en la entrada Toda entrada del usuario, de una API o de otro sistema se considera no confiable hasta que se valide. Ejemplo: consultas parametrizadas en vez de concatenación de strings.

2. Validar siempre Verificar tipo, longitud, formato, rango y conjunto permitido de valores antes de procesar cualquier dato. Ejemplo: express-validator para validar emails y contraseñas.

3. Sanitizar cuando corresponda Preparar datos para su uso seguro en un contexto específico. Prevenir XSS escapeando HTML. En React, no usar dangerouslySetInnerHTML sin sanitizar.

4. Principio de mínimo privilegio Usar únicamente los permisos necesarios para operar. MongoDB con usuario solo para la BD de la app, Docker con usuarios no root, puertos mínimos.

5. Gestión de secretos Nunca almacenar secretos en código fuente. Usar archivos .env excluidos del control de versiones y gestores de secretos como HashiCorp Vault o AWS Secrets Manager.

6. Manejo seguro de errores No exponer stack traces o detalles del sistema al usuario. Mostrar mensajes genéricos y registrar detalles internamente en logs seguros.

7. Dependencias seguras Escanear dependencias periódicamente (npm audit, pip-audit). Actualizar paquetes con vulnerabilidades. Eliminar dependencias abandonadas. Definir política de actualización.

El control 8.28 está directamente relacionado con la prevención de las vulnerabilidades del **OWASP Top 10**. Cada categoría de vulnerabilidad se mapea a uno o más principios de codificación segura. La prevención comienza en el código, no en la revisión final.

Vulnerabilidad	Nivel	Principio de prevención
Injection	Crítica	Validación y sanitización de entradas, consultas parametrizadas
Broken Access Control	Crítica	Control de acceso por roles, mínimo privilegio
Cryptographic Failures	Alta	Cifrado en reposo y tránsito, hashing con bcrypt/argon2
Security Misconfiguration	Alta	Hardening de servidores, contenedores y servicios
Vulnerable Components	Alta	Escaneo de dependencias, actualización periódica
Authentication Failures	Alta	Tokens con expiración, contraseñas hasheadas

El control 8.29 exige que las pruebas de seguridad formen parte del proceso normal de pruebas del sistema. Los hallazgos deben registrarse, evaluarse, corregirse y volver a verificarse. Los entornos de pruebas deben mantenerse protegidos y segregados.

SAST — Análisis Estático

Analiza el código fuente sin ejecutarlo. Un escáner recorre el código buscando patrones conocidos de vulnerabilidades como SQL Injection, hardcoded secrets y XSS. Detecta fallos temprano, antes de compilar o desplegar. Se integra en el pipeline de CI/CD y en el IDE.

Herramientas: ESLint + plugins de seguridad, Semgrep, SonarQube, Bandit

DAST — Análisis Dinámico

Prueba la aplicación mientras está ejecutándose. Un scanner envía ataques simulados contra la app en funcionamiento y reporta vulnerabilidades como XSS reflejado, SQL Injection y configuraciones débiles. Cubre la superficie de ataque expuesta.

Herramientas: OWASP ZAP, Burp Suite

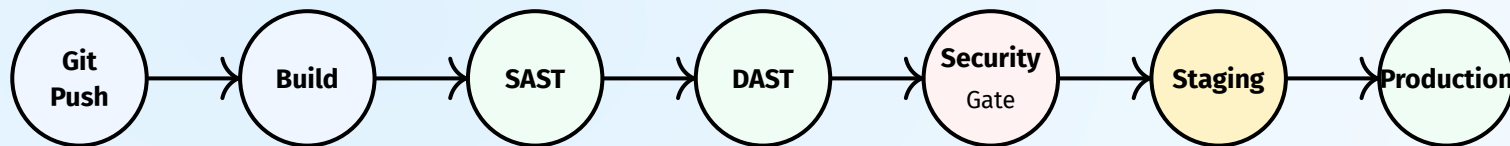
Code Review

Revisión humana del código. Es fundamental porque hay problemas que ninguna herramienta detecta: errores de lógica de negocio, decisiones de diseño inseguras y patrones que el escáner no reconoce. Las revisiones deben ser obligatorias antes de cada merge.

Penetration Testing

Un especialista en seguridad intenta comprometer el sistema de forma controlada. A diferencia del DAST automatizado, demuestra impacto real explotando vulnerabilidades. Es la prueba más cercana a un ataque real y permite validar si los controles implementados son efectivos.

La materialización de los controles 8.25, 8.28 y 8.29 se concreta en un pipeline de integración continua seguro. Cada paso del pipeline aplica un control específico de seguridad.



El **Security Gate** verifica criterios de aceptación: 0 vulnerabilidades críticas, 0 altas, cobertura SAST $\geq 80\%$, dependencias sin CVEs críticos. Si no se cumplen, el pipeline se detiene y no se despliega.

El acceptance testing es la verificación final antes de la producción. Define criterios claros y objetivos que el software debe cumplir. Si no se cumplen, no se despliega. Este es el principio fundamental del control 8.29.

Criterios de aceptación:

- **0** vulnerabilidades críticas
- **0** vulnerabilidades altas
- Cobertura SAST $\geq$ 80%
- Dependencias sin CVEs críticos
- Pruebas ejecutadas y aprobadas
- Configuración endurecida

Si no cumple, no se despliega.

Los hallazgos identificados durante las pruebas deben:

1. **Registrarse** en un sistema de seguimiento
2. **Evaluarte** según su severidad y impacto
3. **Corregirse** antes de avanzar al siguiente paso
4. **Re-verificarse** tras la corrección

Los entornos de pruebas deben estar protegidos y segregados de producción.

La guía de trabajo práctico exige proponer métricas cuantitativas con sus ecuaciones. Estas métricas permiten evaluar objetivamente el cumplimiento de los controles 8.25, 8.28 y 8.29.

Métrica	Fórmula	Objetivo
Densidad de vulnerabilidades	$D_v = \frac{V}{\text{KLOC}}$	$D_v < 1$
Cobertura de análisis SAST	$C = \frac{A_{\text{esc}}}{A_{\text{tot}}} \times 100$	$C \geq 80\%$
Tiempo medio de remediación	$T_{\text{rem}} = \frac{\sum \Delta t}{n}$	$T_{\text{rem}} \leq 7 \text{ días}$
Tasa de aceptación de seguridad	$R = \frac{P_{\text{apr}}}{P_{\text{tot}}} \times 100$	$R = 100\%$
Nivel de madurez SSDLC	$M = \frac{\sum P_i w_i}{\sum w_i}$	$\text{Nivel} \geq 2$

Workcodile-dev es una plataforma web full-stack para estudiantes de la Universidad Nacional de Moquegua. Permite crear, compartir y gestionar cursos y publicaciones con un foro académico. Utiliza un stack tecnológico estándar que representa bien las aplicaciones web modernas.

Stack tecnológico:

- **Frontend:** React 18, TypeScript, Vite, Tailwind CSS, shadcn/ui
- **Backend:** Node.js, Express.js, Mongoose ODM
- **Base de datos:** MongoDB 7
- **Almacenamiento:** MinIO (compatible S3)
- **Infraestructura:** Docker, Docker Compose, Nginx

Autenticación:

- JWT con expiración
- bcrypt para hashing de contraseñas
- Roles: student, moderator, admin

Métricas del proyecto:

- Densidad de vulnerabilidades: $D_v = 0.6$ (Bajo)
- Cobertura SAST: $C_{\text{SAST}} = 88.9\%$
- Tiempo medio de remediación: $T_{\text{rem}} = 5$ días
- Tasa de aceptación: $R = 100\%$
- Madurez SSDLC: Nivel 2 (Gestionado)

Un análisis del código fuente reveló problemas reales de seguridad en la configuración del proyecto. Estos hallazgos demuestran por qué el control 8.25 es necesario: sin un proceso formal de revisión, estos problemas pasan desapercibidos.

Hallazgo	Ubicación	Riesgo
JWT_SECRET hardcodeado como “WorkcoWord”	docker-compose.yml:67	Cualquiera con acceso al repo puede firmar tokens válidos
Credenciales MinIO por defecto (minioadmin)	docker-compose.yml:23-24	Acceso no autorizado al almacenamiento de archivos
MongoDB puerto 27017 expuesto al host	docker-compose.yml:7	Acceso directo a la base de datos desde el host

Estos problemas se habrían evitado con un proceso formal de revisión de configuración antes del despliegue, tal como lo exige el control 8.25.

Un auditor de ISO/IEC 27001 no busca una herramienta específica. Busca un **proceso definido, aplicado y demostrable**. Las evidencias que espera encontrar incluyen:

Evidencias de 8.25:

- Política de Secure SDLC documentada
- Capacitación del equipo en seguridad
- Requisitos de seguridad definidos antes del desarrollo
- Threat modeling realizado y documentado
- Control de cambios y pipeline seguro

Evidencias de 8.28 y 8.29:

- Principios de codificación segura aprobados
- Revisiones de código con hallazgos y correcciones
- Resultados de SAST y DAST
- Criterios de aceptación documentados
- Pipeline CI/CD con Security Gate configurado

¿Preguntas?

Equipo 5 — IS-722 Calidad de Software

Universidad Nacional de Moquegua — 2026-I